PARTIE 3

Mise en œuvre d'une API REST

avec Java et Spring Boot

Cours de Bachelor 3 — Développement Back-end

Spring Boot 4.1.1 · Java 25

Durée	Jours	Prérequis
7 h	**Jour 2 + Jour 3 matin**	**Parties 1 & 2**

1. Introduction

Objectifs de cette partie

À la fin de cette partie, vous serez capables de :

- Créer un projet Spring Boot from scratch avec Spring Initializr.
- Comprendre la structure d'un projet Spring Boot.
- Créer des contrôleurs REST avec les annotations Spring.
- Gérer les 4 méthodes HTTP (GET, POST, PUT, DELETE).
- Connecter l'API à une base de données H2 en mémoire.
- Structurer le code en couches (Controller → Service → Repository).
- Tester l'API avec Postman et curl.
- Expliquer le rôle du conteneur IoC et utiliser l'injection de dépendances par constructeur.
- Décrire le parcours d'une requête dans le framework (filtres, routage, intercepteurs, contrôleur, gestion des erreurs).
- Écrire un test unitaire de service grâce à l'injection de dépendances (Mockito).

Durée estimée

- Théorie et démonstrations : 3 h 30
- Pratique : 3 h 30
- Répartition : Jour 2 complet + Jour 3 matin (voir le planning v2)

Prérequis

- Avoir suivi les Parties 1 et 2.
- Java 25 installé.
- Un IDE (IntelliJ IDEA, VS Code avec Extension Pack for Java, ou Eclipse).
- Maven installé (ou utiliser le wrapper Maven intégré).

---

2. Concepts théoriques essentiels

2.1 Qu'est-ce que Spring Boot ?

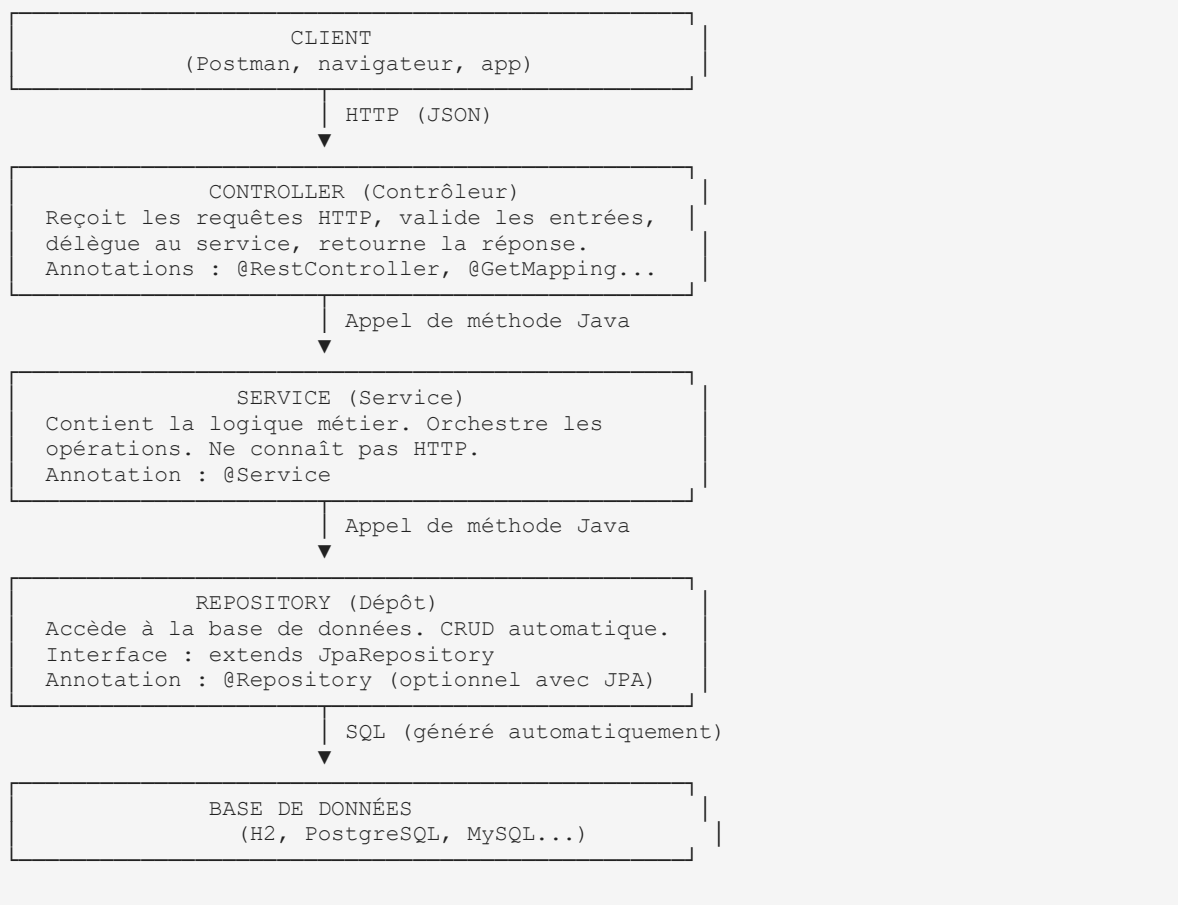
Spring Boot est un framework Java qui simplifie la création d'applications Spring. Il fournit :

- Une **configuration automatique** : pas besoin de fichiers XML interminables.
- Un **serveur embarqué** (Tomcat) : votre API démarre en une commande.
- Des **starters** : des dépendances pré-configurées (web, data, security, etc.).
- Un **écosystème riche** : Spring Data, Spring Security, Spring Cloud, etc.

Avant Spring Boot : configurer un projet Spring demandait des dizaines de fichiers XML et une connaissance approfondie du framework. **Avec Spring Boot :** une annotation `@SpringBootApplication` et c'est parti.

2.2 Architecture en couches

Une API REST bien structurée suit le principe de **séparation des responsabilités**. Chaque couche a un rôle précis :



Pourquoi cette séparation ?

- Le **Controller** ne contient pas de logique métier → plus facile à tester.
- Le **Service** ne connaît pas HTTP → réutilisable dans d'autres contextes.
- Le **Repository** isole l'accès aux données → on peut changer de base de données sans toucher au reste.

2.3 Les annotations Spring essentielles

Annotation	Rôle	Où ?
`@SpringBootApplication`	Point d'entrée de l'application	Classe principale
`@RestController`	Déclare un contrôleur REST (réponses JSON)	Classe contrôleur
`@RequestMapping`	Préfixe d'URL pour tout le contrôleur	Classe contrôleur
`@GetMapping`	Mappe une requête GET	Méthode
`@PostMapping`	Mappe une requête POST	Méthode
`@PutMapping`	Mappe une requête PUT	Méthode
`@DeleteMapping`	Mappe une requête DELETE	Méthode
`@PathVariable`	Extrait une variable de l'URL	Paramètre de méthode
`@RequestBody`	Désérialise le corps JSON en objet Java	Paramètre de méthode

Annotation	Rôle	Où ?
`@RequestParam`	Extrait un paramètre de requête (?clé=valeur)	Paramètre de méthode
`@Service`	Déclare un service (logique métier)	Classe service
`@Repository`	Déclare un dépôt (accès données)	Interface repository
`@Entity`	Déclare une entité JPA (table en base)	Classe modèle
`@Id`	Clé primaire	Champ de l'entité
`@GeneratedValue`	Génération automatique de l'ID	Champ de l'entité

2.4 La base de données H2

H2 est une base de données relationnelle légère, écrite en Java, qui fonctionne **en mémoire**. Elle est parfaite pour le développement et les tests :

- Aucune installation requise.
- Démarre et s'arrête avec l'application.
- Console web intégrée pour visualiser les données.
- Compatible SQL standard.

Attention : les données sont perdues à chaque redémarrage de l'application (mode in-memory). C'est voulu pour le développement.

2.5 Le conteneur IoC et l'injection de dépendances

Le problème : le couplage fort

Imaginons un service qui crée lui-même ses dépendances :

```
public class ProduitService {

    // ❌ Le service fabrique lui-même son repository
    private final ProduitRepository produitRepository = new ProduitRepositoryJdbc();

    // ...
}
```

Ce code pose trois problèmes. Le service est **lié à une implémentation précise** : pour passer de JDBC à JPA, il faut modifier le service. Il est **impossible à tester isolément** : on ne peut pas remplacer le repository par un faux objet (mock). Enfin, **chaque classe gère le cycle de vie de ses dépendances** : si dix services ont besoin du repository, on crée dix instances.

L'inversion de contrôle (IoC)

Avec Spring, ce n'est plus la classe qui crée ses dépendances : c'est le **conteneur IoC** (*Inversion of Control*), appelé `ApplicationContext`, qui crée les objets et les « branche » entre eux. On parle d'**injection de dépendances**. Les objets gérés par le conteneur s'appellent des **beans**.

```
Démarrage : SpringApplication.run(...)
      |
      ▼
1. @ComponentScan parcourt le package com.formation.apirestdemo et ses sous-packages
```

```
   → il trouve les classes annotées @RestController, @Service, @Repository,
@Configuration...
      │
      ▼
2. Le conteneur enregistre une « définition de bean » pour chacune
      │
      ▼
3. Il crée les beans dans l'ordre de leurs dépendances :
   ProduitRepository ──injecté dans──► ProduitService ──injecté dans──►
ProduitController
      │
      ▼
4. Tous les beans sont prêts : Tomcat démarre et écoute sur le port 8080
```

Le service se contente alors de **déclarer** ce dont il a besoin, dans son constructeur :

```
@Service
public class ProduitService {

    private final ProduitRepository produitRepository;

    // ☑ Le conteneur fournit l'implémentation au moment de la création du bean
    public ProduitService(ProduitRepository produitRepository) {
        this.produitRepository = produitRepository;
    }
}
```

Comment déclarer un bean ?

Moyen	Quand l'utiliser	Exemple
Annotations de stéréotype : `@Component`, `@Service`, `@Repository`, `@RestController`	Vos propres classes, détectées automatiquement par le scan	`@Service public class ProduitService`
Méthode `@Bean` dans une classe `@Configuration`	Classes que vous ne pouvez pas annoter (bibliothèques externes) ou qui demandent une construction particulière	`Clock`, `RestClient`, `JsonMapper` (Jackson 3) personnalisé
Auto-configuration	Beans créés automatiquement par Spring Boot selon les starters présents	`DataSource`, `EntityManager`, Jackson, Tomcat

Exemple de bean déclaré avec `@Bean` : une horloge injectable, qui permettra de tester facilement du code dépendant de la date du jour (par exemple le calcul des retards d'emprunt dans le projet fil rouge).

```
package com.formation.apirestdemo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Clock;

@Configuration
public class AppConfig {

    /**
     * Horloge de l'application. En test, on pourra la remplacer par Clock.fixed(...)
     * pour simuler n'importe quelle date.
     */
    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();
    }
```

```
}


@Service
public class EmpruntService {

    private final Clock clock;

    public EmpruntService(Clock clock) {
        this.clock = clock;
    }

    public boolean estEnRetard(LocalDate dateRetourPrevue) {
        return LocalDate.now(clock).isAfter(dateRetourPrevue);
    }
}
```

Les trois façons d'injecter une dépendance

Type d'injection	Exemple	Avis
Par constructeur	`public ProduitService(ProduitRepository repo)`	✅ **Recommandée** : champs `final`, dépendances visibles, classe testable sans Spring
Par setter	`@Autowired public void setRepo(ProduitRepository repo)`	Acceptable pour une dépendance **optionnelle**
Par champ	`@Autowired private ProduitRepository repo;`	❌ À éviter : dépendances cachées, test impossible sans réflexion ou sans démarrer Spring

> **Bon à savoir :** depuis Spring 4.3, lorsqu'une classe n'a **qu'un seul constructeur**, l'annotation `@Autowired` est inutile : Spring l'utilise automatiquement.

Plusieurs implémentations d'une même interface

L'intérêt principal de l'injection de dépendances est de **programmer contre une interface** (principe DIP, voir `Ressources-Complément/SOLID-Principes.md`) et de laisser le conteneur choisir l'implémentation.

```
public interface NotificationService {
    void notifier(String destinataire, String message);
}


@Service
@Primary // implémentation choisie par défaut
public class ConsoleNotificationService implements NotificationService {

    @Override
    public void notifier(String destinataire, String message) {
        // Remplacé par un logger dans la Partie 4b
        System.out.println("[NOTIFICATION] " + destinataire + " : " + message);
    }
}


@Service("emailNotification")
public class EmailNotificationService implements NotificationService {

    @Override
    public void notifier(String destinataire, String message) {
```

```
        // Envoi réel d'un e-mail (non implémenté dans ce cours)
    }
}


@Service
public class CommandeService {

    private final NotificationService notificationService;

    // @Qualifier force une implémentation précise, même si une autre est @Primary
    public CommandeService(@Qualifier("emailNotification") NotificationService
notificationService) {
        this.notificationService = notificationService;
    }
}
```

Situation	Résultat
Deux implémentations, ni `@Primary` ni `@Qualifier`	❌ Erreur au démarrage : `NoUniqueBeanDefinitionException`
Une implémentation marquée `@Primary`	✅ Elle est injectée par défaut
`@Qualifier("nom")` sur le paramètre	✅ L'implémentation nommée est injectée
`@Profile("dev")` / `@Profile("prod")` sur chaque implémentation	✅ L'implémentation dépend de l'environnement (voir Partie 4, §2.10)

> **Erreur fréquente :** `Parameter 0 of constructor in ... required a bean of type '...' that could not be found.` Le plus souvent, la classe n'est pas annotée (`@Service` oublié) ou elle se trouve **en dehors** du package de la classe principale, donc hors du scan.

La portée (scope) d'un bean

Par défaut, un bean est un **singleton** : une seule instance est partagée par **toutes les requêtes**, traitées en parallèle par plusieurs threads.

Portée	Nombre d'instances	Usage
`singleton` (défaut)	Une seule pour toute l'application	Contrôleurs, services, repositories
`prototype`	Une nouvelle à chaque injection	Objets avec état, rarement utile en API REST
`request`	Une par requête HTTP	Données propres à une requête (rare)

> **Conséquence importante :** ne stockez **jamais** de donnée propre à une requête (utilisateur courant, panier, compteur...) dans un champ d'un service ou d'un contrôleur. Deux requêtes simultanées se mélangeraient.

Pourquoi c'est important : la testabilité

Comme le service reçoit son repository par le constructeur, on peut lui donner un **faux repository** (un *mock*) dans un test unitaire. Le test s'exécute en quelques millisecondes, **sans démarrer Spring ni la base de données**. Mockito et AssertJ sont déjà inclus dans `spring-boot-starter-test`.

Créez `src/test/java/com/formation/apirestdemo/service/ProduitServiceTest.java` :

```
package com.formation.apirestdemo.service;

import com.formation.apirestdemo.model.Produit;
import com.formation.apirestdemo.repository.ProduitRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ProduitServiceTest {

    @Mock                    // faux repository, piloté par le test
    private ProduitRepository produitRepository;

    @InjectMocks             // vrai service, construit avec le mock
    private ProduitService produitService;

    @Test
    void recupererParId_retourneLeProduit_quandIlExiste() {
        Produit produit = new Produit();
        produit.setId(1L);
        produit.setNom("Clavier");
        when(produitRepository.findById(1L)).thenReturn(Optional.of(produit));

        Optional<Produit> resultat = produitService.recupererParId(1L);

        assertThat(resultat).isPresent();
        assertThat(resultat.get().getNom()).isEqualTo("Clavier");
    }

    @Test
    void supprimer_retourneFalse_etNeSupprimeRien_siLeProduitNExistePas() {
        when(produitRepository.existsById(99L)).thenReturn(false);

        boolean resultat = produitService.supprimer(99L);

        assertThat(resultat).isFalse();
        verify(produitRepository, never()).deleteById(any());
    }
}
```

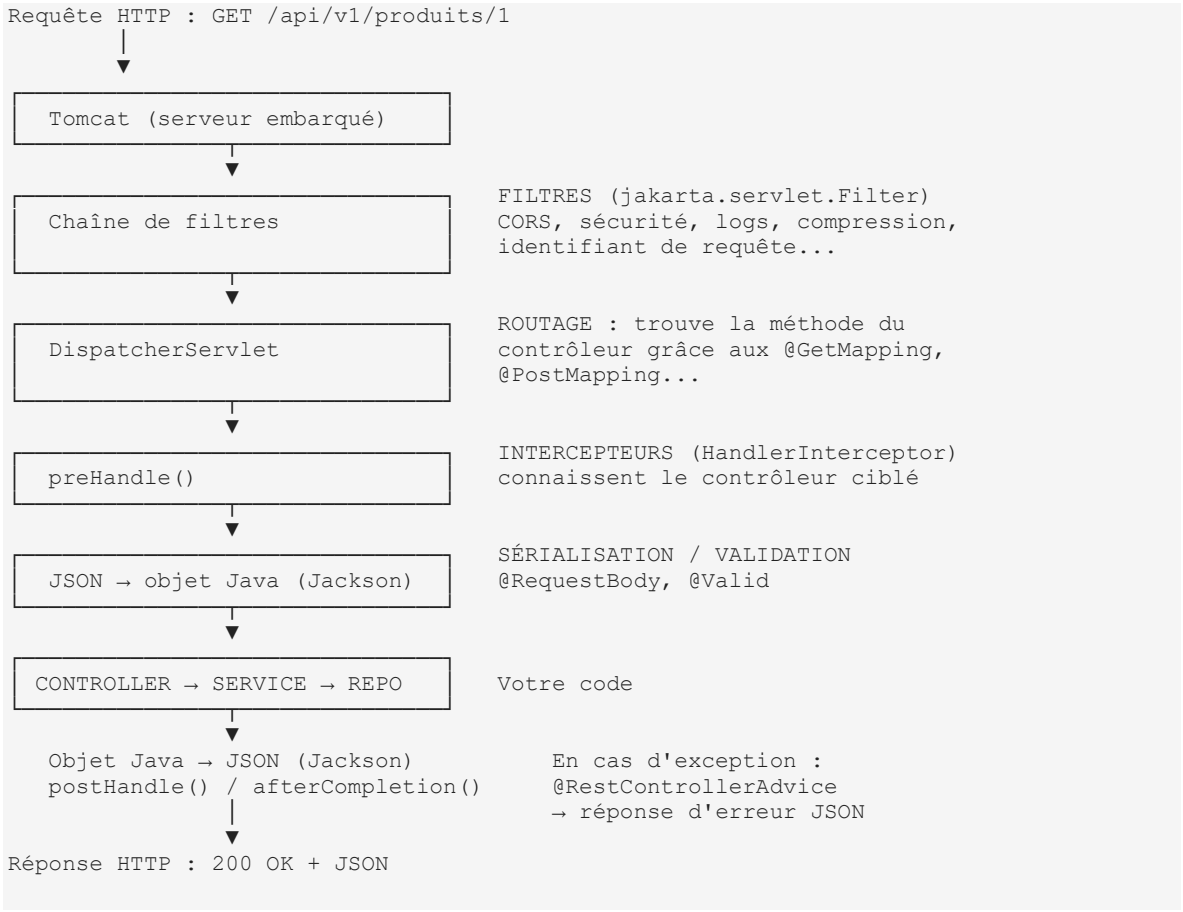
Lancez les tests avec `./mvnw test` (ou le bouton ▶ de l'IDE).

> Les bibliothèques de test sont apportées par les starters `spring-boot-starter-*-test` du `pom.xml` (§3.2). Sur Java 21 et plus, Mockito peut afficher un avertissement sur le chargement dynamique d'un agent Java : il n'empêche pas les tests de passer.

2.6 Le parcours d'une requête HTTP dans Spring Boot

Les « middlewares » de Spring

Dans la plupart des frameworks back-end, un **middleware** est un composant qui s'exécute **avant et/ou après** le traitement d'une requête, pour toutes les routes ou pour un groupe de routes : journalisation, sécurité, CORS, compression, mesure du temps de réponse... Dans Spring Boot, ce rôle est tenu par trois mécanismes : les **filtres**, les **intercepteurs** et le **gestionnaire global d'exceptions**.



Mécanisme	Niveau	Connaît le contrôleur appelé ?	Cas d'usage typiques
Filtre (`Filter`, `OncePerRequestFilter`)	Servlet, avant Spring MVC	Non	Logs de toutes les requêtes, CORS, sécurité, compression, identifiant de corrélation
Intercepteur (`HandlerInterceptor`)	Spring MVC	Oui	Mesure du temps d'un endpoint, contrôle d'un en-tête, audit
`@RestControllerAdvice`	Spring MVC	Oui (exception levée)	Transformer les exceptions en réponses JSON homogènes

Exemple : un intercepteur qui mesure la durée des requêtes

```
package com.formation.apirestdemo.config;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class DureeInterceptor implements HandlerInterceptor {

    private static final String DEBUT = "debutTraitement";

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response,
                             Object handler) {
        // On stocke l'heure de début dans la requête, PAS dans un champ :
        // l'intercepteur est un singleton partagé par toutes les requêtes.
        request.setAttribute(DEBUT, System.currentTimeMillis());
        return true; // false = la requête s'arrête ici, le contrôleur n'est pas
appelé
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse
response,
                                Object handler, Exception ex) {
        long duree = System.currentTimeMillis() - (long) request.getAttribute(DEBUT);
        // Remplacé par un logger dans la Partie 4b
        System.out.println(request.getMethod() + " " + request.getRequestURI()
                + " → " + response.getStatus() + " (" + duree + " ms)");
    }
}
```

L'intercepteur doit ensuite être **enregistré** dans la configuration de Spring MVC. Remarquez qu'il est lui-même injecté par constructeur :

```
package com.formation.apirestdemo.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final DureeInterceptor dureeInterceptor;

    public WebConfig(DureeInterceptor dureeInterceptor) {
        this.dureeInterceptor = dureeInterceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(dureeInterceptor)
                .addPathPatterns("/api/**"); // uniquement les routes de l'API
    }
}
```

Un exemple de **filtre** (journalisation de toutes les requêtes avec un identifiant de corrélation) est construit dans la **Partie 4b**.

Récapitulatif : les briques d'un framework back-end et leur équivalent Spring Boot

Brique	Rôle	Dans Spring Boot
Routage	Associer une URL + une méthode HTTP à du code	`DispatcherServlet` + `@RequestMapping`, `@GetMapping`...
Contrôleurs	Recevoir la requête, déléguer, construire la réponse	`@RestController`
Services	Porter la logique métier	`@Service`
Accès aux données	Lire et écrire en base	`@Repository`, Spring Data JPA
Configuration	Paramétrer le framework sans modifier le code	`application.properties`, `@Configuration` + `@Bean`, auto-configuration, profils
Middlewares	Traitements transverses avant/après la requête	`Filter`, `HandlerInterceptor`
Gestion des erreurs	Transformer les exceptions en réponses propres	`@RestControllerAdvice` + `@ExceptionHandler`
Sérialisation	Convertir objets ⇄ JSON	Jackson (configurable, voir Partie 4 §2.9)
Validation	Refuser les entrées invalides	Jakarta Bean Validation : `@Valid`, `@NotBlank`...
Conteneur IoC	Créer et assembler les objets	`ApplicationContext`

Ces briques existent dans tous les frameworks back-end modernes, sous d'autres noms :

Framework	Démarrage rapide	Routage / contrôleurs	Middlewares	Injection de dépendances
Spring Boot (Java)	start.spring.io, `spring init`	`@RestController`, `@GetMapping`	`Filter`, `HandlerInterceptor`	Conteneur IoC intégré
NestJS (TypeScript)	`nest new`	`@Controller`, `@Get`	Middlewares, intercepteurs, pipes	Conteneur intégré (`@Injectable`)
ASP.NET Core (C#)	`dotnet new webapi`	Contrôleurs ou *minimal APIs*	Pipeline `app.Use...`	Conteneur intégré (`builder.Services`)
Express (Node.js)	`npx express-generator`	`app.get(...)`, `Router`	`app.use(...)`	Aucun conteneur par défaut

3. Mise en pratique guidée

3.1 Créer le projet Spring Boot

> **Versions utilisées : Spring Boot 4.1.1 et Java 25.** Spring Boot 4 apporte des changements visibles dans ce cours :.

Méthode 1 : Spring Initializr (recommandée)

1. Allez sur https://start.spring.io
2. Configurez le projet :

Paramètre	Valeur
Project	Maven
Language	Java
Spring Boot	4.1.1
Group	`com.formation`
Artifact	`api-rest-demo`
Name	`api-rest-demo`
Package name	`com.formation.apirestdemo`
Packaging	Jar
Java	25

3. Ajoutez les dépendances :
   - **Spring Web** : pour créer des contrôleurs REST (ajoute `spring-boot-starter-webmvc`)
   - **Spring Data JPA** : pour l'accès aux données
   - **H2 Database** : base de données en mémoire
   - **Spring Boot DevTools** : rechargement automatique en développement
   - **Validation** : annotations `@NotBlank`, `@Positive`, `@Valid`...
4. Cliquez sur **Generate** → téléchargez le zip → décompressez → ouvrez dans votre IDE.
5. Ouvrez le `pom.xml` généré et comparez-le à celui du §3.2. Ajoutez si besoin la dépendance `spring-boot-h2console` (console H2) et les starters de test manquants.

Méthode 2 : en ligne de commande (alternative)

```
# Si vous avez Spring Boot CLI installé :
spring init --boot-version=4.1.1 \
  --dependencies=web,data-jpa,h2,devtools,validation \
  --group-id=com.formation \
  --artifact-id=api-rest-demo \
  --java-version=25 \
  api-rest-demo.zip
```

3.2 Le fichier pom.xml

Voici le `pom.xml` complet avec toutes les dépendances nécessaires :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <!-- Hérite de Spring Boot : fournit les versions de toutes les dépendances -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>

    <groupId>com.formation</groupId>
    <artifactId>api-rest-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>api-rest-demo</name>
    <description>API REST de démonstration pour la formation</description>

    <properties>
        <java.version>25</java.version>
    </properties>

    <dependencies>
        <!-- Spring MVC : contrôleurs REST, Jackson 3, serveur Tomcat embarqué
             (ce starter s'appelait spring-boot-starter-web avant Spring Boot 4) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <!-- Spring Data JPA : accès aux données, repositories automatiques
(Hibernate 7) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- Validation : @NotBlank, @Positive, @Valid... -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- H2 : base de données en mémoire pour le développement -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Console web H2 : module séparé depuis Spring Boot 4.
             Sans lui, spring.h2.console.enabled=true n'a aucun effet. -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-h2console</artifactId>
        </dependency>

        <!-- DevTools : redémarrage automatique en développement (exclu du jar final)
-->
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!-- Tests : depuis Spring Boot 4, un starter de test par technologie.
             Chacun inclut spring-boot-starter-test (JUnit 5, Mockito, AssertJ). -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

> **Ce qui a changé avec Spring Boot 4** : Spring Boot est désormais découpé en **modules plus petits**. Un projet n'embarque que ce dont il a besoin : c'est plus léger, mais il faut ajouter explicitement certains modules (comme la console H2) et le starter de test de chaque technologie utilisée.

3.3 Structure du projet

Après génération, votre projet a cette structure :

```
api-rest-demo/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/formation/apirestdemo/
│   │   │       ├── ApiRestDemoApplication.java      ← Point d'entrée
│   │   │       ├── model/
│   │   │       │   └── Produit.java                 ← Entité JPA
│   │   │       ├── repository/
│   │   │       │   └── ProduitRepository.java       ← Accès données
│   │   │       ├── service/
│   │   │       │   └── ProduitService.java          ← Logique métier
│   │   │       └── controller/
│   │   │           └── ProduitController.java       ← Endpoints REST
│   │   └── resources/
│   │       ├── application.properties               ← Configuration
│   │       └── data.sql                             ← Données initiales (optionnel)
│   └── test/
│       └── java/
│           └── com/formation/apirestdemo/
```

```
│               └── ApiRestDemoApplicationTests.java
```

3.4 Configuration : application.properties

```
# Nom de l'application
spring.application.name=api-rest-demo

# Port du serveur (8080 par défaut)
server.port=8080

# --- Configuration H2 ---
# URL de la base de données en mémoire
spring.datasource.url=jdbc:h2:mem:formationdb
# Driver JDBC
spring.datasource.driver-class-name=org.h2.Driver
# Identifiants
spring.datasource.username=sa
spring.datasource.password=

# --- Console H2 (accessible via navigateur) ---
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# --- JPA / Hibernate ---
# Créer les tables automatiquement à partir des entités
spring.jpa.hibernate.ddl-auto=create-drop
# Afficher les requêtes SQL dans la console (utile en développement)
spring.jpa.show-sql=true
# Formater les requêtes SQL pour la lisibilité
spring.jpa.properties.hibernate.format_sql=true
# Remarque : Hibernate 7 détecte automatiquement le dialecte H2, inutile de le
préciser
```

3.5 Étape 1 : La classe principale

```
package com.formation.apirestdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Point d'entrée de l'application Spring Boot.
 * L'annotation @SpringBootApplication active :
 * - @Configuration : classe de configuration Spring
 * - @EnableAutoConfiguration : configuration automatique
 * - @ComponentScan : détecte les composants dans ce package et ses sous-packages
 */
@SpringBootApplication
public class ApiRestDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(ApiRestDemoApplication.class, args);
    }
}
```

Lancez l'application :

```
# Avec Maven Wrapper (recommandé)
./mvnw spring-boot:run

# Ou avec Maven installé globalement
mvn spring-boot:run
```

Vous devriez voir dans la console :

```
Started ApiRestDemoApplication in 2.5 seconds
Tomcat started on port 8080
```

Testez dans le navigateur : `http://localhost:8080` → vous obtiendrez une page d'erreur 404 (Whitelabel Error Page). C'est normal : nous n'avons pas encore créé de contrôleur.

3.6 Étape 2 : Le modèle (Entity)

Créez le fichier `src/main/java/com/formation/apirestdemo/model/Produit.java` :

```
package com.formation.apirestdemo.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Column;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

/**
 * Entité JPA représentant un produit.
 * Chaque instance correspond à une ligne dans la table "produits".
 */
@Entity
@Table(name = "produits")
public class Produit {

    /**
     * Identifiant unique, généré automatiquement par la base de données.
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /**
     * Nom du produit. Ne peut pas être vide.
     */
    @NotBlank(message = "Le nom du produit est obligatoire")
    @Column(nullable = false)
    private String nom;

    /**
     * Description du produit.
     */
    private String description;

    /**
     * Prix du produit. Doit être positif.
     */
    @Positive(message = "Le prix doit être positif")
    @Column(nullable = false)
    private Double prix;

    /**
     * Quantité en stock.
     */
    private Integer stock;

    // --- Constructeurs ---

    /**
     * Constructeur sans arguments (requis par JPA).
```

```
     */
    public Produit() {
    }

    /**
     * Constructeur avec tous les champs (sauf l'id, généré automatiquement).
     */
    public Produit(String nom, String description, Double prix, Integer stock) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
        this.stock = stock;
    }

    // --- Getters et Setters ---

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public Double getPrix() {
        return prix;
    }

    public void setPrix(Double prix) {
        this.prix = prix;
    }

    public Integer getStock() {
        return stock;
    }

    public void setStock(Integer stock) {
        this.stock = stock;
    }

    @Override
    public String toString() {
        return "Produit{id=" + id + ", nom='" + nom + "', prix=" + prix + "}";
    }
}
```

Points clés :

- `@Entity` indique à JPA que cette classe correspond à une table en base.

- `@Id` + `@GeneratedValue` : la clé primaire est auto-générée.
- `@NotBlank` et `@Positive` : validation des données (utilisée plus tard avec `@Valid`).
- Le constructeur sans arguments est **obligatoire** pour JPA.

3.7 Étape 3 : Le Repository

Créez `src/main/java/com/formation/apirestdemo/repository/ProduitRepository.java` :

```
package com.formation.apirestdemo.repository;

import com.formation.apirestdemo.model.Produit;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Repository pour l'entité Produit.
 * En héritant de JpaRepository, Spring génère automatiquement
 * les méthodes CRUD : findAll(), findById(), save(), deleteById(), etc.
 *
 * Pas besoin d'écrire l'implémentation : Spring Data le fait pour nous.
 */
@Repository
public interface ProduitRepository extends JpaRepository<Produit, Long> {

    /**
     * Recherche des produits dont le nom contient la chaîne donnée (insensible à la
casse).
     * Spring Data génère automatiquement la requête SQL à partir du nom de la
méthode.
     * Équivalent SQL : SELECT * FROM produits WHERE LOWER(nom) LIKE LOWER('%nom%')
     */
    List<Produit> findByNomContainingIgnoreCase(String nom);

    /**
     * Recherche des produits dont le prix est inférieur ou égal au montant donné.
     * Équivalent SQL : SELECT * FROM produits WHERE prix <= ?
     */
    List<Produit> findByPrixLessThanEqual(Double prix);
}
```

C'est la magie de Spring Data JPA. En déclarant simplement une interface qui étend `JpaRepository<Produit, Long>`, vous obtenez gratuitement :

Méthode	Action
`findAll()`	Retourne tous les produits
`findById(Long id)`	Retourne un produit par son ID
`save(Produit p)`	Crée ou met à jour un produit
`deleteById(Long id)`	Supprime un produit par son ID
`count()`	Compte le nombre de produits
`existsById(Long id)`	Vérifie si un produit existe

3.8 Étape 4 : Le Service

Créez `src/main/java/com/formation/apirestdemo/service/ProduitService.java` :

```
package com.formation.apirestdemo.service;

import com.formation.apirestdemo.model.Produit;
import com.formation.apirestdemo.repository.ProduitRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

/**
 * Service contenant la logique métier pour les produits.
 * Le service est le chef d'orchestre : il coordonne les appels
 * au repository et applique les règles métier.
 */
@Service
public class ProduitService {

    private final ProduitRepository produitRepository;

    /**
     * Injection de dépendance par constructeur.
     * Spring injecte automatiquement le ProduitRepository.
     */
    public ProduitService(ProduitRepository produitRepository) {
        this.produitRepository = produitRepository;
    }

    /**
     * Retourne tous les produits.
     */
    public List<Produit> recupererTous() {
        return produitRepository.findAll();
    }

    /**
     * Retourne un produit par son ID.
     * @return Optional vide si le produit n'existe pas.
     */
    public Optional<Produit> recupererParId(Long id) {
        return produitRepository.findById(id);
    }

    /**
     * Crée un nouveau produit.
     * @param produit Le produit à créer (sans ID).
     * @return Le produit créé (avec ID généré).
     */
    public Produit creer(Produit produit) {
        // On s'assure que l'ID est null pour forcer une création (pas une mise à
jour)
        produit.setId(null);
        return produitRepository.save(produit);
    }

    /**
     * Met à jour un produit existant.
     * @param id L'ID du produit à modifier.
     * @param produitModifie Les nouvelles valeurs.
     * @return Optional contenant le produit mis à jour, ou vide si l'ID n'existe
pas.
     */
    public Optional<Produit> mettreAJour(Long id, Produit produitModifie) {
        return produitRepository.findById(id)
                .map(produitExistant -> {
                    produitExistant.setNom(produitModifie.getNom());
```

```
                    produitExistant.setDescription(produitModifie.getDescription());
                    produitExistant.setPrix(produitModifie.getPrix());
                    produitExistant.setStock(produitModifie.getStock());
                    return produitRepository.save(produitExistant);
                });
    }

    /**
     * Supprime un produit par son ID.
     * @return true si le produit existait et a été supprimé, false sinon.
     */
    public boolean supprimer(Long id) {
        if (produitRepository.existsById(id)) {
            produitRepository.deleteById(id);
            return true;
        }
        return false;
    }

    /**
     * Recherche des produits par nom (recherche partielle, insensible à la casse).
     */
    public List<Produit> rechercherParNom(String nom) {
        return produitRepository.findByNomContainingIgnoreCase(nom);
    }
}
```

Points clés :

- `@Service` déclare cette classe comme un bean Spring géré par le conteneur.
- L'injection se fait **par constructeur** (meilleure pratique, pas de `@Autowired` nécessaire).
- Le service retourne des `Optional` pour gérer proprement les cas « introuvable ».
- La méthode `mettreAJour` utilise `map()` sur l'Optional pour enchaîner les opérations.

3.9 Étape 5 : Le Controller

Créez `src/main/java/com/formation/apirestdemo/controller/ProduitController.java` :

```
package com.formation.apirestdemo.controller;

import com.formation.apirestdemo.model.Produit;
import com.formation.apirestdemo.service.ProduitService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * Contrôleur REST pour les produits.
 * Chaque méthode correspond à un endpoint de l'API.
 *
 * @RestController = @Controller + @ResponseBody
 * → Toutes les méthodes retournent du JSON (pas des vues HTML).
 */
@RestController
@RequestMapping("/api/v1/produits")
public class ProduitController {

    private final ProduitService produitService;

    public ProduitController(ProduitService produitService) {
```

```
        this.produitService = produitService;
    }

    // ============================================================
    // GET /api/v1/produits — Lister tous les produits
    // ============================================================
    @GetMapping
    public ResponseEntity<List<Produit>> listerTous() {
        List<Produit> produits = produitService.recupererTous();
        return ResponseEntity.ok(produits); // 200 OK
    }

    // ============================================================
    // GET /api/v1/produits/{id} — Obtenir un produit par son ID
    // ============================================================
    @GetMapping("/{id}")
    public ResponseEntity<Produit> obtenirParId(@PathVariable Long id) {
        return produitService.recupererParId(id)
                .map(ResponseEntity::ok)                          // 200 OK si trouvé
                .orElse(ResponseEntity.notFound().build());       // 404 si non
trouvé
    }

    // ============================================================
    // POST /api/v1/produits — Créer un nouveau produit
    // ============================================================
    @PostMapping
    public ResponseEntity<Produit> creer(@Valid @RequestBody Produit produit) {
        Produit produitCree = produitService.creer(produit);
        return ResponseEntity
                .status(HttpStatus.CREATED) // 201 Created
                .body(produitCree);
    }

    // ============================================================
    // PUT /api/v1/produits/{id} — Mettre à jour un produit
    // ============================================================
    @PutMapping("/{id}")
    public ResponseEntity<Produit> mettreAJour(@PathVariable Long id,
                                               @Valid @RequestBody Produit produit)
{
        return produitService.mettreAJour(id, produit)
                .map(ResponseEntity::ok)                          // 200 OK si mis à
jour
                .orElse(ResponseEntity.notFound().build());       // 404 si non
trouvé
    }

    // ============================================================
    // DELETE /api/v1/produits/{id} — Supprimer un produit
    // ============================================================
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> supprimer(@PathVariable Long id) {
        if (produitService.supprimer(id)) {
            return ResponseEntity.noContent().build(); // 204 No Content
        }
        return ResponseEntity.notFound().build();      // 404 si non trouvé
    }

    // ============================================================
    // GET /api/v1/produits/recherche?nom=clavier — Rechercher par nom
    // ============================================================
    @GetMapping("/recherche")
    public ResponseEntity<List<Produit>> rechercherParNom(@RequestParam String nom) {
        List<Produit> resultats = produitService.rechercherParNom(nom);
        return ResponseEntity.ok(resultats);
    }
}
```

Décortiquons les annotations :

- `@RestController` : combine `@Controller` et `@ResponseBody`. Chaque méthode retourne du JSON.
- `@RequestMapping("/api/v1/produits")` : préfixe commun pour toutes les routes de ce contrôleur.
- `@GetMapping("/{id}")` : mappe les requêtes `GET /api/v1/produits/42`.
- `@PathVariable Long id` : extrait le `42` de l'URL et le convertit en `Long`.
- `@RequestBody Produit produit` : désérialise le JSON du corps en objet `Produit`.
- `@Valid` : déclenche la validation des annotations (`@NotBlank`, `@Positive`).
- `ResponseEntity<T>` : permet de contrôler le code de statut, les en-têtes et le corps de la réponse.

3.10 Étape 6 : Données initiales (optionnel)

Pour avoir des données dès le démarrage, créez `src/main/resources/data.sql` :

```
-- Insertion de données de test au démarrage de l'application
INSERT INTO produits (nom, description, prix, stock) VALUES
('Clavier mécanique', 'Clavier gaming RGB avec switches Cherry MX', 89.99, 50),
('Souris ergonomique', 'Souris sans fil avec support poignet', 45.50, 120),
('Écran 27 pouces', 'Moniteur IPS 4K pour développeurs', 349.00, 25),
('Casque audio', 'Casque Bluetooth avec réduction de bruit', 129.99, 75),
('Hub USB-C', 'Hub 7 ports avec HDMI et lecteur SD', 39.99, 200),
('Webcam HD', 'Webcam 1080p avec micro intégré', 59.99, 90),
('Tapis de souris XXL', 'Tapis gaming 80x30cm antidérapant', 19.99, 300),
('Support laptop', 'Support réglable en aluminium', 34.99, 60);
```

Important : Spring Boot exécute automatiquement `data.sql` au démarrage si le fichier est dans `src/main/resources/`. Ajoutez cette ligne dans `application.properties` :

```
spring.jpa.defer-datasource-initialization=true
```

Cela garantit que les tables sont créées avant l'exécution du script SQL.

3.11 Étape 7 : Lancer et tester

Démarrer l'application

```
./mvnw spring-boot:run
```

Tester avec curl

```
# Lister tous les produits
curl -s http://localhost:8080/api/v1/produits | python3 -m json.tool

# Obtenir le produit n°1
curl -s http://localhost:8080/api/v1/produits/1 | python3 -m json.tool

# Créer un nouveau produit
curl -X POST http://localhost:8080/api/v1/produits \
  -H "Content-Type: application/json" \
  -d '{
    "nom": "Câble HDMI 2.1",
    "description": "Câble 2 mètres, 8K compatible",
    "prix": 15.99,
    "stock": 500
```

```
  }'

# Modifier un produit
curl -X PUT http://localhost:8080/api/v1/produits/1 \
  -H "Content-Type: application/json" \
  -d '{
    "nom": "Clavier mécanique PRO",
    "description": "Version améliorée avec repose-poignets",
    "prix": 109.99,
    "stock": 40
  }'

# Supprimer un produit
curl -X DELETE http://localhost:8080/api/v1/produits/1

# Rechercher par nom
curl -s "http://localhost:8080/api/v1/produits/recherche?nom=souris" | python3 -m
json.tool
```

Tester avec Postman

Créez une collection "API Produits" dans Postman avec les requêtes suivantes :

#	Méthode	URL	Corps
1	GET	`http://localhost:8080/api/v1/produits`	—
2	GET	`http://localhost:8080/api/v1/produits/1`	—
3	POST	`http://localhost:8080/api/v1/produits`	JSON du nouveau produit
4	PUT	`http://localhost:8080/api/v1/produits/1`	JSON du produit modifié
5	DELETE	`http://localhost:8080/api/v1/produits/1`	—
6	GET	`http://localhost:8080/api/v1/produits/recherche?nom=clavier`	—

Accéder à la console H2

Ouvrez `http://localhost:8080/h2-console` dans le navigateur :

- JDBC URL : `jdbc:h2:mem:formationdb`
- User Name : `sa`
- Password : *(laisser vide)*

Cliquez sur **Connect**. Vous pouvez exécuter des requêtes SQL directement :

```
SELECT * FROM produits;
SELECT COUNT(*) FROM produits;
```

3.12 Gestion des erreurs

Pour retourner des messages d'erreur propres au lieu du stack trace par défaut, ajoutez un gestionnaire d'exceptions global.

Créez `src/main/java/com/formation/apirestdemo/exception/GlobalExceptionHandler.java` :

```
package com.formation.apirestdemo.exception;

import org.springframework.http.HttpStatus;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

/**
 * Gestionnaire global des exceptions.
 * Intercepte les exceptions levées par les contrôleurs
 * et retourne des réponses JSON structurées.
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    /**
     * Gère les erreurs de validation (@Valid).
     * Retourne un 400 avec la liste des champs en erreur.
     */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(
            MethodArgumentNotValidException ex) {

        Map<String, String> erreurs = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(erreur ->
                erreurs.put(erreur.getField(), erreur.getDefaultMessage())
        );

        Map<String, Object> reponse = new HashMap<>();
        reponse.put("timestamp", LocalDateTime.now().toString());
        reponse.put("statut", 400);
        reponse.put("erreur", "Validation échouée");
        reponse.put("details", erreurs);

        return ResponseEntity.badRequest().body(reponse);
    }

    /**
     * Gère toutes les autres exceptions non prévues.
     * Retourne un 500 avec un message générique.
     */
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneral(Exception ex) {

        Map<String, Object> reponse = new HashMap<>();
        reponse.put("timestamp", LocalDateTime.now().toString());
        reponse.put("statut", 500);
        reponse.put("erreur", "Erreur interne du serveur");
        reponse.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(reponse);
    }
}
```

Testez la validation :

```
# Envoyer un produit sans nom → doit retourner 400
curl -X POST http://localhost:8080/api/v1/produits \
  -H "Content-Type: application/json" \
  -d '{ "nom": "", "prix": -5 }'
```

Réponse attendue :

```
{
  "timestamp": "2025-09-15T14:30:00",
```

```
  "statut": 400,
  "erreur": "Validation échouée",
  "details": {
    "nom": "Le nom du produit est obligatoire",
    "prix": "Le prix doit être positif"
  }
}
```

---

4. Exercices pratiques

Exercice 1 — Hello World REST (Facile)

Objectif : Créer votre premier contrôleur Spring Boot.

Consigne :

1. Créez un nouveau contrôleur `BonjourController` mappé sur `/api/v1/bonjour`.
2. Implémentez les endpoints suivants :

Méthode	URL	Comportement
GET	`/api/v1/bonjour`	Retourne `{"message": "Bonjour le monde !"}`
GET	`/api/v1/bonjour/{prenom}`	Retourne `{"message": "Bonjour Marie !"}`
POST	`/api/v1/bonjour`	Accepte `{"prenom": "Jean"}` et retourne `{"message": "Bienvenue Jean !"}`

3. Testez chaque endpoint avec curl ou Postman.

Conseil : Pour retourner du JSON simple, vous pouvez utiliser une `Map<String, String>`.

```
@GetMapping
public Map<String, String> bonjour() {
    return Map.of("message", "Bonjour le monde !");
}
```

Exercice 2 — CRUD Utilisateurs (Intermédiaire)

Objectif : Reproduire l'architecture complète (Model → Repository → Service → Controller) pour une nouvelle entité.

Consigne : Ajoutez au projet existant la gestion des **utilisateurs** :

Modèle Utilisateur :

Champ	Type	Contrainte
id	Long	Auto-généré
nom	String	Obligatoire
prenom	String	Obligatoire
email	String	Obligatoire, unique
age	Integer	Positif

Travail demandé :

1. Créez la classe `Utilisateur` avec les annotations JPA et de validation.
2. Créez l'interface `UtilisateurRepository` avec une méthode `findByEmail`.
3. Créez la classe `UtilisateurService` avec toutes les opérations CRUD.
4. Créez la classe `UtilisateurController` avec les endpoints :

Méthode	URL	Action
GET	`/api/v1/utilisateurs`	Lister tous
GET	`/api/v1/utilisateurs/{id}`	Obtenir par ID
POST	`/api/v1/utilisateurs`	Créer
PUT	`/api/v1/utilisateurs/{id}`	Modifier
DELETE	`/api/v1/utilisateurs/{id}`	Supprimer

5. Ajoutez des données initiales dans `data.sql`.
6. Testez tous les endpoints avec Postman.

Travail en binôme : Un étudiant code le modèle et le repository, l'autre code le service et le controller. Intégrez et testez ensemble.

Exercice 3 — Fonctionnalités avancées (Avancé)

Objectif : Enrichir l'API produits avec des fonctionnalités réalistes.

Consigne : Ajoutez au contrôleur `ProduitController` les endpoints suivants :

1. **Recherche multi-critères :**

```
GET /api/v1/produits/filtrer?prixMin=10&prixMax=100&nom=clavier
```

Tous les paramètres sont optionnels. Ajoutez la méthode dans le repository en utilisant `@Query` :

```
@Query("SELECT p FROM Produit p WHERE " +
       "(:nom IS NULL OR LOWER(p.nom) LIKE LOWER(CONCAT('%', :nom, '%'))) AND " +
       "(:prixMin IS NULL OR p.prix >= :prixMin) AND " +
       "(:prixMax IS NULL OR p.prix <= :prixMax)")
List<Produit> filtrer(@Param("nom") String nom,
                      @Param("prixMin") Double prixMin,
                      @Param("prixMax") Double prixMax);
```

2. **Statistiques :**

```
GET /api/v1/produits/statistiques
```

Retourne :

```
{
  "nombreTotal": 8,
  "prixMoyen": 96.18,
  "prixMin": 19.99,
  "prixMax": 349.00,
  "stockTotal": 920
}
```

3. **Mise à jour du stock uniquement :**

```
PATCH /api/v1/produits/{id}/stock?quantite=10
```

Ajoute la quantité donnée au stock actuel du produit.

Travail en binôme : Chaque étudiant implémente une fonctionnalité, puis faites une revue de code croisée.

Exercice 4 — Injection de dépendances, tests et intercepteur (Intermédiaire)

Objectif : Manipuler le conteneur IoC, en mesurer le bénéfice sur la testabilité et écrire un premier « middleware ».

Consigne :

1. **Deux implémentations d'une interface.** Créez l'interface `NotificationService` et ses deux implémentations `ConsoleNotificationService` et `EmailNotificationService` (§2.5). Injectez `NotificationService` dans `ProduitService` et appelez `notifier("stock", "Nouveau produit : " + nom)` dans la méthode `creer`.
2. **Observer le conteneur.** Retirez `@Primary` et relancez l'application : relevez le message d'erreur exact. Corrigez-le **sans** remettre `@Primary`.
3. **Tester grâce à l'injection.** Complétez `ProduitServiceTest` avec deux nouveaux tests :
   - `creer` force l'ID à `null` avant la sauvegarde (utilisez un `ArgumentCaptor<Produit>`) ;
   - `creer` appelle bien `notificationService.notifier(...)` une seule fois (ajoutez un second `@Mock`).
4. **Écrire un intercepteur.** Enregistrez `DureeInterceptor` (§2.6), puis créez un second intercepteur `CleApiInterceptor` qui refuse les requêtes `DELETE` ne contenant pas l'en-tête `X-Api-Key: formation` : il renvoie `401` et `preHandle` retourne `false`.

Tests attendus :

```
# Sans clé → 401
curl -i -X DELETE http://localhost:8080/api/v1/produits/1

# Avec clé → 204
curl -i -X DELETE http://localhost:8080/api/v1/produits/1 -H "X-Api-Key: formation"
```

Questions de réflexion :

- Pourquoi le test unitaire de `ProduitService` n'a-t-il pas besoin de démarrer Spring ?
- Pourquoi `DureeInterceptor` stocke-t-il l'heure de début dans la requête et non dans un champ de la classe ?
- `CleApiInterceptor` est-il une vraie solution de sécurité ? Quelle solution utiliseriez-vous en production ?

Travail en binôme : Un étudiant écrit les implémentations et l'intercepteur, l'autre écrit les tests. Échangez ensuite les rôles pour la revue de code.

---

5. Résumé des points clés

Ce qu'il faut retenir :

- **Spring Boot** simplifie la création d'API REST grâce à la configuration automatique et au serveur Tomcat embarqué.
- L'architecture en **3 couches** (Controller → Service → Repository) sépare les responsabilités et facilite la maintenance.
- Les annotations `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` mappent les méthodes HTTP sur des méthodes Java.
- `@PathVariable` extrait les variables de l'URL, `@RequestBody` désérialise le JSON, `@RequestParam` lit les query parameters.
- **Spring Data JPA** génère automatiquement les requêtes CRUD à partir d'une simple interface.
- **H2** est une base de données en mémoire idéale pour le développement.
- `ResponseEntity` permet de contrôler finement le code de statut et le corps de la réponse.
- `@Valid` + `@RestControllerAdvice` permettent de valider les entrées et de retourner des erreurs structurées.
- Le **conteneur IoC** (`ApplicationContext`) crée les beans et leur injecte leurs dépendances. L'injection **par constructeur** rend le code découplé et testable sans démarrer Spring.
- `@Primary`, `@Qualifier` et `@Profile` permettent de choisir entre plusieurs implémentations d'une même interface.
- Une requête traverse **filtres** → **DispatcherServlet** → **intercepteurs** → **contrôleur** ; les exceptions remontent au `@RestControllerAdvice`. Filtres et intercepteurs sont les « middlewares » de Spring.

Checklist avant de passer à la suite :

- ☐ Mon projet Spring Boot démarre sans erreur.
- ☐ Je sais créer un contrôleur avec les 4 méthodes CRUD.
- ☐ Je comprends le rôle de chaque couche (Controller, Service, Repository).
- ☐ Je sais tester mes endpoints avec Postman et curl.
- ☐ Je sais accéder à la console H2 pour vérifier les données.
- ☐ Je sais expliquer ce qu'est un bean et qui le crée.
- ☐ Je sais résoudre une ambiguïté entre deux implémentations (`@Primary` / `@Qualifier`).
- ☐ Je sais écrire un test unitaire de service avec Mockito.
- ☐ Je sais écrire et enregistrer un intercepteur.

6. Pour aller plus loin

> Les pistes d'approfondissement de cette partie sont regroupées dans le document `Aller-Plus-Loin/03-Aller-Plus-Loin-Spring-Boot.md` :
>
> - Projections Spring Data et Spring Data REST
> - Documentation automatique avec `springdoc-openapi` (traitée en Partie 4)
> - Cycle de vie des beans (`@PostConstruct`, `@PreDestroy`) et beans conditionnels
> - Comprendre l'auto-configuration (rapport de conditions)
> - Tests d'intégration de la couche web (`@WebMvcTest`, `@SpringBootTest`)

Prochaine partie → Bonnes pratiques pour l'élaboration d'une API REST : DTOs, pagination, gestion d'erreurs avancée, documentation, sécurité, sérialisation et profils.